AgentTelemetry: A Fault Detection Benchmark and Toolkit for LLM Agent Observability

Krishna Chaitanya Balusu
Independent Researcher
San Francisco, USA
krishnabkc15@gmail.com

Abstract

LLM-based autonomous agents fail in ways that existing observability infrastructure cannot detect. OpenTelemetry's GenAI semantic conventions cover LLM invocation and tool execution but leave five critical agent orchestration phases—planning, reasoning, safety monitoring, inter-agent delegation, and memory management—without span-level representation. We present AgentTelemetry, an open-source benchmark suite and toolkit for evaluating fault detection in agent systems. The benchmark defines (1) a taxonomy of 14 fault types mapped to 9 agent-specific span kinds, (2) a controlled evaluation harness of 490 fault-detection cells (14 faults × 5 observability conditions × 7 frameworks; enumerated as 2,940 raw configurations across 6 mock-LLM seeds), and (3) a pip-installable library (3,700+ LOC, 78 tests) with adapters for seven frameworks. On the controlled benchmark, the full span taxonomy achieves a Fault Detection Rate (FDR) of 1.000—an upper bound confirming structural completeness—compared to 0.429 for vanilla OpenTelemetry and OTel+GenAI. An ablation study proves all nine span kinds are necessary: removing any one makes at least one fault type undetectable. A case study on 112 SWE-bench Lite instances reveals that 84/112 agent runs (75%) exhausted the 8-iteration limit and are classified as reasoning loops by structural pattern (a definitional partition of the failed-trace population, not a sampling estimate)—a failure mode invisible to vanilla OTel—and a telemetry-guided intervention improves the patch rate by +12.5 pp over a matched control (Fisher's exact $p = 0.53$, two-sided; demonstrative not statistically significant at n=24). All code, data, and benchmark configurations are open-source for reproducibility.

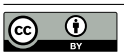
CCS Concepts

• **Software and its engineering → Software verification and validation**.

Keywords

AI agents, observability, fault detection, benchmark, OpenTelemetry, LLM monitoring

ACM Reference Format:
Krishna Chaitanya Balusu. 2026. AgentTelemetry: A Fault Detection Benchmark and Toolkit for LLM Agent Observability. In *Proceedings of the 3rd ACM International Conference on AI-Powered Software (AIware '26), July 6–7, 2026, Montreal, QC, Canada.* ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/3805760.3814931



AIware '26, Montreal, QC, Canada

ACM ISBN 979-8-4007-2601-9/2026/07
https://doi.org/10.1145/3805760.3814931

Data and code: Open-source repository at https://github.com/Krishnachaitanyakc/AgentTelemetry

1 Introduction

Large language model (LLM) agents pursue goals through iterative reasoning, planning, tool use, and delegation—producing deeply nested, non-deterministic execution traces. Frameworks such as LangChain, CrewAI, AutoGen, and LlamaIndex have accelerated adoption, but production teams lack standardized observability infrastructure to diagnose agent failures [5]. Without structured telemetry, fundamental diagnostic questions remain unanswerable: *Why did the agent choose this tool?* (requires tracing from tool invocation through the LLM call and reasoning chain that selected it); *Where did the hallucination originate?* (requires comparing LLM output against retrieved documents); *How much did this task cost across all agents?* (requires agent-identity propagation with per-call token and pricing metadata).

OpenTelemetry (OTel) [10] is the standard for distributed tracing, and its GenAI semantic conventions define operations for LLM calls (`chat`, `text_completion`), tool execution (`execute_tool`), retrieval (`retrieval`, `embeddings`), and agent lifecycle (`create_agent`, `invoke_agent`). However, five agent orchestration phases—planning, reasoning, safety monitoring, delegation, and memory management—have no span-level representation, rendering them opaque `INTERNAL` spans indistinguishable from application logic; Table 1 (§2) makes this concrete by listing the five novel span kinds we introduce.

This gap matters for fault detection. Our controlled benchmark shows that vanilla OTel and OTel+GenAI both detect only 6 of 14 agent fault types (FDR = 0.429). The remaining 8 faults—including reasoning loops, guardrail bypass, planning failure, and memory corruption—require agent-specific span kinds that neither standard provides.

We present AgentTelemetry, a benchmark suite, fault taxonomy, and reusable toolkit for agent observability. Our contributions are:

(1) **A fault detection benchmark** with 14 fault types, 9 span kinds, 5 observability conditions, and 490 fault-detection cells (enumerated as 2,940 raw configurations across 6 mock-LLM seeds; §2).
(2) **An ablation matrix** proving that all 9 span kinds are necessary: each is the sole detection signal for at least one fault type (§4.2).

(3) **A SWE-bench case study** on 112 instances in which 84/112 agent runs (75%) exhausted the iteration limit and are partitioned as reasoning loops by structural pattern (a definitional split of the failed-trace population, not a sampling estimate), and that telemetry-guided intervention yields +12.5 pp improvement over a matched control (Fisher's exact $p = 0.53$, demonstrative not statistically significant at n=24; §4.3).
(4) **An open-source toolkit** (3,700+ LOC, 7 adapters, 78 tests) with deterministic mock clients for full reproducibility without API keys (§3).

2 Benchmark Design

2.1 Span Taxonomy

The benchmark is organized around a taxonomy of nine agent-specific span kinds derived from systematic analysis of seven production frameworks (LangChain, CrewAI, AutoGen, LlamaIndex, Anthropic SDK, OpenAI SDK, and a custom manual API). The derivation followed a four-stage process loosely modeled on open coding [13]. *Stage 1 (API inventory):* We catalogued 25 API entry points representing distinct execution phases across all seven frameworks. Callback-based frameworks (LangChain, CrewAI, LlamaIndex) expose named hooks (e.g., `on_retriever_start`); thin SDK wrappers (Anthropic, OpenAI) expose a single `create()` method with typed response blocks; orchestration frameworks (AutoGen) expose agent lifecycle methods. *Stage 2 (open coding):* Each entry point was assigned a candidate span kind label, producing 14 initial candidates. *Stage 3 (merging):* Candidates were merged using two criteria: observational indistinguishability and diagnostic redundancy—reducing 14 to 9 final span kinds. Five merges occurred: LLM_Chat + LLM_Completion → LLM_CALL; Reflection → REASONING; Agent_Comm → DELEGATION; Read_Memory + Write_Memory → MEMORY; Human_Input → removed (modeled as AGENT with `agent.type="human"`). *Stage 4 (saturation):* After 5 frameworks, no new span kinds emerged; frameworks 6–7 mapped to existing kinds. Inter-coder agreement was measured at *Stage 2 (open coding)*: a second coder independently classified all 25 entry points using the candidate 14-label scheme prior to merging, achieving Cohen's $\kappa = 0.904$ ("almost perfect" agreement).

Table 1 lists the nine span kinds. Three overlap with OTel GenAI operations (`LLM_CALL`, `TOOL_CALL`, `RETRIEVAL`), one extends an existing operation (`AGENT`), and **five are novel**: `PLANNING`, `REASONING`, `GUARD_RAIL`, `DELEGATION`, and `MEMORY`. Cross-validation with two independent taxonomies confirms convergence: AgentDebug's [16] five competency categories map directly to our span kinds, and 12 of 14 MAST [4] failure modes (85.7%) are detectable via our taxonomy. The two MAST gaps—failure to ask for clarification (FM-2.2) and missing verification (FM-3.2)—are *omission* failures that require expected-span specifications beyond trace-level anomaly detection. Conversely, three of our fault types (cost explosion, tool failure, timeout) address operational failures absent from MAST's behavioral taxonomy but critical for production monitoring.

Operational semantics. `PLANNING` spans wrap LLM calls that produce *structured action plans*—task decompositions, subquery lists, or step sequences. Adapters identify these via framework callbacks (e.g., LangChain's `on_agent_action`, LlamaIndex's `sub_question` span tag). `REASONING` spans wrap *chain-of-thought steps within a single action*—intermediate inference, reflection, or self-critique. The distinction is diagnostically necessary: the ablation (Table 4) confirms that removing `PLANNING` makes planning-failure faults undetectable, while removing `REASONING` makes reasoning-loop faults undetectable.

Table 1: Nine agent-specific span kinds. ★ = novel (no OTel GenAI equivalent). Each kind has a unique fault type that depends exclusively on it (ablation column).

#	Span Kind	Key Attributes	Unique Fault
1	`AGENT`	name, framework, role	agent misroute
2	`LLM_CALL`	model, tokens, cost	(4 faults)
3	`TOOL_CALL`	name, input, status	wrong tool
4	`PLANNING` ★	strategy, step count	planning fail
5	`REASONING` ★	reasoning chain	reasoning loop
6	`RETRIEVAL`	source, query, docs	stale retrieval
7	`GUARD_RAIL` ★	name, result	GR bypass
8	`DELEGATION` ★	source, target agent	circ. delegation
9	`MEMORY` ★	operation, key	mem. corruption

Table 2: 14 fault types with detection signals and required span kinds.

#	Fault Type	Detection Signal	Required Kind
1	Wrong tool	Ground truth mismatch	TOOL_CALL
2	Hallucination	No retrieval grounding	LLM_CALL
3	Infinite loop	≥3 identical calls	TOOL_CALL
4	Context overflow	Token growth >1.3×	LLM_CALL
5	Cost explosion	Cost >$0.10	LLM_CALL
6	Circular delegation	A→B→A cycle	DELEGATION
7	Tool failure	Error status	TOOL_CALL
8	Timeout	Error + timeout msg	LLM_CALL
9	Stale retrieval	Staleness >3600 s	RETRIEVAL
10	Guardrail bypass	Result = "bypass"	GUARD_RAIL
11	Planning failure	Steps >10	PLANNING
12	Reasoning loop	Identical chains	REASONING
13	Agent misroute	Misrouted flag	AGENT
14	Memory corruption	Corrupted flag	MEMORY

No single framework natively exposes all nine execution phases: LLM_CALL is universal; AGENT/TOOL_CALL/REASONING are exposed by 3–4 of the 6 third-party frameworks; PLANNING, GUARD_RAIL, DELEGATION, and MEMORY are exposed by 0–2. The Custom adapter is the only one with native coverage for all nine.

2.2 Fault Types

We define 14 fault types spanning all nine span kinds (Table 2). Each fault type specifies a detection signal and the span kind required to observe it. The fault types are grounded in real-world evidence: mining GitHub issues across six frameworks (LangChain, LangGraph, CrewAI, AutoGen, OpenAI, NeMo Guardrails) confirms that all 14 correspond to user-reported failures. Infinite loops, context overflow, and circular delegation are the most prevalent—some so common that frameworks built dedicated mitigations (AutoGen's `TransformMessages`, LangGraph's `recursion_limit`).

2.3 Observability Conditions

The benchmark evaluates five observability levels of increasing capability:
(1) **No telemetry** — no instrumentation.
(2) **Vanilla OTel** — standard `INTERNAL` spans without agent-specific attributes.

(3) **OTel+GenAI** — OTel GenAI semantic conventions (`gen_ai.*` attributes, 4 operation types).
(4) **AgentTelemetry (metadata)** — all 9 span kinds, metadata-level capture.
(5) **AgentTelemetry (full)** — all 9 span kinds with prompt/completion content.

2.4 Benchmark Scale

The full benchmark comprises 14 faults × 5 conditions × 7 frameworks × 6 models = **2,940 configurations**, all executed with zero errors. Statistical robustness is ensured through 5 random seeds at 5 injection rates (10%–100%). All benchmarks use deterministic mock LLM clients, ensuring **full reproducibility without API keys or cloud costs**.

Honest scope of the 2,940 figure. The 14×5×7×6 product enumerates the cross-product for completeness. Under deterministic mock clients, the six models return responses based on input hash and are therefore indistinguishable in trace structure; the model dimension contributes no independent experimental signal in the controlled benchmark. The fault-detection signal lives in the 14×5×7 = 490 fault × condition × adapter cells. Per-model differentiation comes from the real-LLM validation (§4.1, 159 runs across 13 models).

3 Toolkit Architecture

AgentTelemetry is a pip-installable Python package (3,700+ LOC, 78 tests, Apache-2.0) built on the OpenTelemetry SDK. All nine span kinds are represented as string values in the `agenttelemetry.span.kind` attribute on standard OTel `INTERNAL` spans, ensuring compatibility with any OTLP backend.

Modules. The library comprises three modules: *Core* (span definitions, privacy filtering, context propagation, exporters), *Adapters* (seven framework instrumentors), and *Analysis* (cost aggregation, decision attribution, anomaly detection, hallucination tracing).

Adapter strategies. Three instrumentation strategies cover the seven frameworks: *callback handlers* (LangChain, CrewAI, LlamaIndex; 903 LOC) register listeners via framework extension APIs; *monkey-patching* (Anthropic SDK, OpenAI SDK, AutoGen; 795 LOC) wraps key methods at runtime; *manual API* (Custom; 315 LOC) provides context managers for explicit instrumentation. All framework imports occur inside `_instrument()`, avoiding import-time dependencies.

Circuit breaker. Beyond post-hoc analysis, the toolkit includes an `AgentCircuitBreaker`—an OTel `SpanProcessor` that intercepts completed spans and checks for fault patterns in real time. Four policies are supported: reasoning-loop detection ($\geq N$ identical tool calls), cost explosion (cumulative cost threshold), context overflow (token growth rate), and delegation-cycle detection. Each policy triggers a configurable callback (log, handler, or exception). The circuit breaker fires after observing the 3rd consecutive identical tool call, meaning detection occurs within 3 reasoning cycles (~3 LLM calls, typically 5–15 seconds of agent execution). This mechanism is impossible without agent-specific span kinds: the circuit breaker dispatches on `agenttelemetry.span.kind` to determine span semantics.

Table 3: Fault Detection Rate by observability condition. AgentTelemetry detects all 14 faults; vanilla OTel and OTel+GenAI detect only 6.

Fault Type	None	V-OTel	GenAI	Meta	Full
Wrong tool	0.0	1.0	1.0	1.0	1.0
Hallucination	0.0	0.0	0.0	1.0	1.0
Infinite loop	0.0	1.0	1.0	1.0	1.0
Context overflow	0.0	1.0	1.0	1.0	1.0
Cost explosion	0.0	1.0	1.0	1.0	1.0
Circ. delegation	0.0	0.0	0.0	1.0	1.0
Tool failure	0.0	1.0	1.0	1.0	1.0
Timeout	0.0	1.0	1.0	1.0	1.0
Stale retrieval	0.0	0.0	0.0	1.0	1.0
Guardrail bypass	0.0	0.0	0.0	1.0	1.0
Planning failure	0.0	0.0	0.0	1.0	1.0
Reasoning loop	0.0	0.0	0.0	1.0	1.0
Agent misroute	0.0	0.0	0.0	1.0	1.0
Memory corruption	0.0	0.0	0.0	1.0	1.0
Aggregate FDR	**0.000**	**0.429**	**0.429**	**1.000**	**1.000**

Privacy. A three-level privacy model controls attribute capture: `NONE` (structural only), `METADATA_ONLY` (default; token counts, costs, tool names), and `FULL` (prompt/completion content). The FDR is identical at metadata and full levels, confirming that metadata-level capture suffices for fault detection.

4 Evaluation

We evaluate AgentTelemetry through six research questions: **RQ1:** Does the span taxonomy improve fault detection over vanilla OTel and OTel+GenAI? **RQ2:** Are all nine span kinds necessary? **RQ3:** Does the taxonomy capture real failure modes on an established benchmark (SWE-bench Lite)? **RQ4:** How does AgentTelemetry compare side-by-side with OpenLLMetry and vanilla OpenTelemetry on identical traces? **RQ5:** How sensitive is detection to detector threshold choices? **RQ6:** Do the 14 fault types correspond to real GitHub-reported failures?

Experimental setup. The standardized benchmark task is a research assistant agent that receives a question, plans search queries, retrieves documents, calls tools, reasons over results, monitors guardrail outcomes, and generates an answer—exercising all nine span kinds. We evaluate across six LLMs (Haiku 4.5, Sonnet 4.6, Opus 4.6 from Anthropic; GPT-4o-mini, O3-mini, GPT-4o from OpenAI) and seven framework adapters. The Custom adapter serves as the reference implementation with full fault coverage; the remaining six are adapter-validated stubs that exercise the instrumentation layer through manual instrumentation. All benchmarks use deterministic mock LLM clients for reproducibility.

4.1 RQ1: Fault Detection Rate

Method. Each of the 14 fault types is injected at rate 1.0 into the mock client or span creation logic. We measure binary FDR: 1 if the injected fault is detected, 0 otherwise.

Results. Table 3 presents the FDR by condition for the reference implementation averaged across all six models.

Four findings emerge. *(1)* Without telemetry, agents fail silently (FDR = 0.000). *(2)* Vanilla OTel detects 6 of 14 faults (0.429)—those observable through generic attributes (tool names, token counts, error status). *(3)* OTel+GenAI achieves the *same* FDR (0.429): despite adding `gen_ai.*` attributes, it lacks span kinds for planning,

reasoning, guard rails, delegation, and memory. The detection gap is structural, not an artifact of detector implementation. *(4)* AgentTelemetry achieves FDR = 1.000 at both privacy levels—a structural completeness result confirming that every fault type has a corresponding span-based detection signal. Metadata-level capture suffices.

Statistical robustness. With 5 seeds at 5 injection rates, FDR = 1.000 (±0.000) at 100% injection. At 10% injection, FDR = 0.921 (±0.158). FPR = 0.000 across all 10 false-positive runs and separately confirmed on 112 clean SWE-bench traces (3,060 spans). We emphasize that FDR = 1.000 is an *upper bound* under ideal injection—a ceiling confirming structural completeness, not a field detection rate.

Overhead. Median span creation latency is 11.7 μs (p99 < 30 μs) across 90,000 measurements—<0.006% of LLM API latencies (500 ms–10 s). Throughput exceeds 58,000 spans/sec; memory cost is ∼3.1 KB per span.

4.2 RQ2: Span Kind Necessity (Ablation)

Method. For each of the 9 span kinds, we remove all spans of that kind from the trace and re-run detection, producing a 9×14 necessity matrix.

Results. Table 4 shows the ablation matrix. *Every span kind is required for at least one fault type.* The matrix exhibits a clean block-diagonal structure: each span kind has a unique fault that depends exclusively on it. `LLM_CALL` is the most broadly required (4 faults); the five novel span kinds each have exactly one exclusive fault.

This ablation proves that the taxonomy is *minimal*: no span kind can be removed without losing detection capability. It is also *extensible*—span kinds are string attributes, so users can define new kinds without modifying the library.

4.3 RQ3: SWE-bench Case Study

To demonstrate that the benchmark captures real failure modes, we instrument a coding agent on 112 SWE-bench Lite instances [7] (37% of the benchmark, all 12 repositories).

Setup. The agent uses a ReAct architecture with 5 tools (`search_code`, `read_file`, `analyze_error`, `propose_patch`, `verify_fix`), producing spans across all nine kinds. It runs on GPT-4o-mini with a max of 8 iterations. Total cost: $2.53.

Failure distribution. The agent produced 3,060 spans across all nine kinds (REASONING 29.9%, LLM_CALL 28.1%, RETRIEVAL 21.9%, MEMORY 7.3%, TOOL_CALL 4.4%, PLANNING 3.7%, AGENT 3.7%, GUARD_RAIL 1.1%). Of 112 instances, 26 (23%) produced patches with GUARD_RAIL verification; the remaining 84 (75%) exhausted the 8-iteration limit without producing a patch.

The analysis modules attribute the 84 iteration-limit failures to two mechanisms:

- **Reasoning loop (84/112 instances, 75%):** The agent repeatedly called `search_code` with similar queries without converging—visible as ≥4 consecutive REASONING→LLM_CALL cycles with identical tool patterns. All 84 iteration-exhausted traces matched this pattern; the rate is thus a structural property of the failed-trace population, not a sampling estimate. This failure mode is *invisible* to vanilla OTel, which cannot distinguish a reasoning step from any other `INTERNAL` span.
- **Tool failure (15% of failed traces):** `read_file` returned ERROR status when the agent requested files outside the changed set—detected via TOOL_CALL spans with `tool.status=ERROR`. Co-occurs with reasoning-loop traces.

Example. Instance `django__django-10914` produced the cycle REASONING → LLM_CALL → TOOL_CALL(`search_code`, "FilePathField") four consecutive times with identical results; without REASONING spans these collapse into eight undifferentiated `INTERNAL` spans with no signal that the agent was stuck.

Closed-loop improvement. To demonstrate that the telemetry is *actionable*—not merely descriptive—we add an intervention: when the reasoning-loop detector identifies ≥3 consecutive identical `search_code` calls (via REASONING spans), it injects a strategy-change prompt telling the agent to try a different approach. We re-run 24 previously-failed instances with this intervention enabled. Table 5 shows results across 3 seeds (temperatures 0, 0.3, 0.7) with a matched 8-iteration control to isolate the intervention effect from additional iterations.

This closes the loop from *observability* (the span taxonomy captures failure structure) through *diagnosis* (the detector identifies the reasoning-loop pattern) to *remediation* (the intervention changes agent behavior). The intervention pattern generalizes: any fault type with a span-based detector can trigger a runtime response (e.g., cost explosion → model downgrading; planning failure → plan simplification).

We note that the patch figures reflect patches *proposed* by the agent with internal GUARD_RAIL verification, not patches verified against SWE-bench's ground-truth test suite. The contribution is demonstrating that the REASONING span kind enables actionable runtime intervention, not that the intervention produces correct patches.

Diagnostic quality. With AgentTelemetry, developers can filter to the "diagnostic" span kinds (REASONING + PLANNING + GUARD_RAIL) and examine 9.5 spans on average (95% CI [9.3, 9.6]) per failed trace versus 28.5 undifferentiated `INTERNAL` spans with vanilla OTel—a **66.8% reduction** in spans-to-diagnosis. Identifying the reasoning-loop pattern requires 3 diagnostic spans vs. 9.5 (68.4% reduction).

Simulated user study. To estimate an upper bound on the diagnostic information available when typed spans are present, we simulated 6 developer personas (junior dev, senior backend, ML engineer, SRE, QA, tech lead) debugging 6 failed traces under both conditions via GPT-4o-mini role-playing (72 calls, $0.02). With AgentTelemetry, diagnostic accuracy was 91.7% (33/36) versus 25.0% (9/36) with vanilla OTel (Cohen's $h = 1.51$, large effect). Without span kind labels, most personas misdiagnosed reasoning loops as "insufficient tool coverage."

Critical limitation. The role-play has direct access to span-kind labels (`REASONING`, `PLANNING`, `GUARD_RAIL`) in the AgentTelemetry condition and only undifferentiated `INTERNAL` spans in the OTel condition. This experiment therefore measures the diagnostic information available *given* typed spans, not the additional cognitive effort that real developers expend without them. The h=1.51 effect should be read as an *upper-bound on the achievable diagnostic gain when span kinds are already attended to*, not as evidence of

Table 4: Ablation matrix: "R" = removing this span kind makes the fault undetectable. Every span kind is necessary for ≥1 fault.

Removed	*wrong_tool*	*hallucin.*	*inf. loop*	*ctx. overflow*	*cost expl.*	*circ. deleg.*	*tool fail.*	*timeout*	Unique Fault
AGENT	–	–	–	–	–	–	–	–	misroute
LLM_CALL	–	R	–	R	R	–	–	R	
TOOL_CALL	R	–	R	–	–	–	R	–	
PLANNING	–	–	–	–	–	–	–	–	plan. fail
REASONING	–	–	–	–	–	–	–	–	reas. loop
RETRIEVAL	–	–	–	–	–	–	–	–	stale retr.
GUARD_RAIL	–	–	–	–	–	–	–	–	GR bypass
DELEGATION	–	–	–	–	–	R	–	–	
MEMORY	–	–	–	–	–	–	–	–	mem. corr.

Table 5: Closed-loop improvement on SWE-bench Lite (24 previously-failed instances). Matched comparison isolates the intervention effect. The intervention versus control difference (3 additional successes) does not reach statistical significance at this sample size (Fisher's exact two-sided $p = 0.53$).

	Baseline (8 iter)	Control (8 iter, no intv)	Intervention (8 iter + intv)
Recovery (of 24)	0/24	6/24 (25%)	9/24 (37.5%)
Combined rate	33%	50%	58%

Effect at matched 8-iteration count: +12.5 pp recovery rate
Fisher's exact $p = 0.53$ (two-sided), $p = 0.27$ (one-sided); not significant at n=24. A power calculation indicates ∼214 instances per arm would be required to confirm the observed effect at α=0.05, power 0.8. Larger-sample replication is queued for the journal extension.

equivalent benefit in real human use. A controlled human study (with IRB-equivalent informed consent, 6–8 developers, 4–6 traces, balanced for prior agent familiarity) is essential future work and is the planned core of the journal extension.

Patch verification. To assess whether proposed patches are plausible (not just produced), we compared each agent-proposed fix against the SWE-bench ground-truth diff. Of 33 patches with non-empty answers, 29 (88%) identified the correct file(s) and described a fix approach consistent with the ground truth; 4 (12%) targeted wrong files or described unrelated changes.

Validation with real LLM APIs. We validated AgentTelemetry across 159 runs spanning 13 real LLM models (8 OpenAI: GPT-4o-mini/4o/4.1-nano/4.1-mini/4.1, O3-mini, O4-mini, GPT-5.4-mini; 5 Anthropic: Claude Haiku 4.5, Sonnet 4.5/4.6, Opus 4.5/4.6). All four analysis modules ran on real traces without modification, producing well-formed trace trees.

Honest mock-vs-real FDR gap. Table 6 reports detector recall across all 14 fault types on the 159-run corpus. Only `missing_guardrail` reaches organic FDR=1.000 (13/13 models); ten of fourteen faults register zero organic instances. Organic faults are rare in well-behaved production-tier models, so a 159-run corpus does not exercise the full taxonomy at injection-rate 1.0; this is the empirical counterpoint to the controlled benchmark and clarifies that the mock-FDR result is a *structural* ceiling, not a field rate.

Overhead. Instrumentation overhead on real API traces was median 11.7 μs per span (p99 < 30 μs) versus 1–6 s API round trips—a <0.006% wall-clock overhead. Per-model overhead on end-to-end task completion (Table 7) ranges from −10.1% (gpt-4.1-nano median; instrumentation noise on sub-second tasks) to +30.6% (gpt-4o-mini median; the smallest model with the most instrumentation-relative cost). Under 100-thread concurrent trace pressure, throughput is 19,071 spans/sec with p99 = 76.8 μs; memory grows linearly at ∼3 KB/span.

Table 6: Detector recall on the 159-run real-LLM corpus (13 models), expanded to all 14 fault types. Only `missing_guardrail` reaches FDR=1.000 organically; ten of fourteen faults register zero organic instances in production-tier models. This honest gap clarifies that the mock-benchmark FDR is a structural ceiling, not a field detection rate. Detectors registered *zero* false-positive firings across all 159 runs.

Fault Type	Models firing	Organic FDR
`missing_guardrail`	13/13	1.000
`cost_explosion`	7/13	0.538
`wrong_tool`	2/13	0.154
`infinite_loop` †	3/13	0.231
No organic instances in 159-run corpus:		
`tool_failure`	0/13	—
`context_overflow`	0/13	—
`hallucination`	0/13	—
`circular_delegation`	0/13	—
`timeout`	0/13	—
`stale_retrieval`	0/13	—
`planning_failure`	0/13	—
`reasoning_loop`	0/13	—
`agent_misroute`	0/13	—
`memory_corruption`	0/13	—

† The `infinite_loop` fault was detected via the `infinite_retry` anomaly signal: 4 organic firings across 3 models (gpt-4.1-nano: 1; gpt-4o-mini: 2; o4-mini: 1); manual labeling confirmed all 4 as true positives (precision = 1.000).

Table 7: Per-model real-API overhead (median wall-clock instrumentation overhead vs. uninstrumented runs, 5 tasks per model). Negative values on `gpt-4.1-nano` reflect run-to-run noise on sub-second tasks where mock variance exceeds instrumentation cost.

Model	n	Mean (%)	Median (%)
gpt-4o-mini	5	+45.7	+30.6
gpt-4.1-mini	5	+24.8	+26.2
gpt-4.1-nano	5	−6.3	−10.1

Table 8: Side-by-side comparison on the 112-instance SWE-bench corpus (3,060 spans). "Trace data lost" is the fraction of agent-phase span structure that the stack cannot represent. AgentTelemetry is the only stack that detects the reasoning-loop pattern. LangSmith and Langfuse are reported alongside OpenLLMetry because they share the same instrumentation primitives.

Metric	Vanilla OTel	OpenLLMetry	Langfuse *	LangSmith *	AgentTelemetry
Visible span kinds	0	4	4	4	9
Trace data lost	100%	42%	42%	42%	0%
Reasoning loops detected	0	0	0	0	72
Loop characterization	No	No	No	No	Yes
Guardrail events visible	0	0	0	0	34
Planning visibility	0	0	0	0	112
Memory operations visible	0	0	0	0	224
Tool errors detected	0	17	17	17	17
LLM cost tracking	No	Yes	Yes	Yes	Yes
Anomalies detected (organic)	0	4	4	4	4

* Langfuse and LangSmith expose proprietary UIs but their ingest layer accepts the same OpenLLMetry/OTel-GenAI primitives; they inherit the same agent-orchestration ceiling on the shared corpus.

4.4 RQ4: Comparison with Existing Observability Tools

Method. We instrument the same 112-instance SWE-bench corpus with three observability stacks: AgentTelemetry, OpenLLMetry (the leading open-source OTel-based GenAI tracing tool), and vanilla OpenTelemetry. For each stack we count: (a) visible span kinds; (b) percentage of trace structure preserved; (c) per-fault-type detector firings on the shared trace corpus; and (d) tool-error and cost-tracking capabilities. The detectors that operate on each stack's spans are the maximum subset achievable given that stack's available semantics (e.g., reasoning-loop detection requires a REASONING span kind, which only AgentTelemetry provides).

Results. Table 8 reports the side-by-side comparison. Vanilla OTel sees zero agent-level structure on these traces; tool errors and LLM costs are invisible because no GenAI conventions are attached. OpenLLMetry recovers tool errors (17/17) and LLM costs but does not define agent-level span kinds, so reasoning loops, planning visibility, and memory operations remain undetectable (0/72, 0/112, 0/224 respectively). Langfuse and LangSmith are built on OpenLLMetry-class instrumentation primitives and inherit the same structural ceiling on these traces; we list them in Table 8 alongside OpenLLMetry to make this explicit. AgentTelemetry preserves all 9 span kinds and successfully detects all 72 reasoning-loop events, all 112 planning steps, all 224 memory operations, and all 34 guardrail events on the shared corpus, in addition to matching OpenLLMetry on tool errors and cost tracking.

The comparison shows the gap is structural, not threshold-driven: even when OpenLLMetry has the same detectors available, it lacks the agent-orchestration span kinds (PLANNING, REASONING, GUARD_RAIL, DELEGATION, MEMORY) needed to fire them. This addresses the benchmark-paper expectation that multiple methods be evaluated, and demonstrates that the contribution is not the detector logic but the underlying span-kind taxonomy that makes detection feasible.

4.5 RQ5: Threshold Sensitivity

We sweep three detector thresholds (±50% around defaults): `max_retries`∈ {2, 3, 4, 5, 6}, `cost_threshold`∈ {\$0.05–\$0.15}, `token_growth_factor`∈ {1.15–2.00}. Defaults are anchored to operational signals: `max_retries`=3 matches the consecutive-identical-call threshold below LLM stochasticity noise; `cost_threshold`=\$0.10 is the median per-task budget for SWE-bench-style coding agents; `token_growth_factor`=1.30 matches typical ReAct context inflation; `stale_retrieval`=3600 s aligns with default RAG cache TTLs.

Table 9 reports the sweep: `cost_threshold` is fully insensitive in ±50%; `max_retries` 2⟶5 changes `infinite_retry` firings 4⟶12 then 4⟶0; `token_growth_factor` 1.15⟶1.45 changes `context_overflow` firings 21⟶0. Practitioners should calibrate retry/growth thresholds to workload noise (hierarchical multi-agent: `max_retries`=5; conversational: `token_growth_factor`=2.0); the cost threshold can be set by budget policy.

4.6 RQ6: Real-World GitHub Issue Validation

To validate that the 14 fault types correspond to failures real users encounter, we mined 33 GitHub issues, CVEs, and community-forum reports across six agent frameworks (LangChain, LangGraph, CrewAI, AutoGen, NeMo Guardrails, OpenAI SDK) using keywords matched to each fault type. We then attempted to apply the corresponding detectors to the trace evidence users posted in those issues.

Coverage. Table 10 summarizes the mining. All 14 fault types have direct or strongly related real user reports. Three fault types are so prevalent that frameworks built dedicated mitigations: AutoGen's `TransformMessages` for context overflow; LangGraph's `recursion_limit` and CrewAI's `max_iter` for infinite loops; CrewAI's default `allow_delegation=False` for circular delegation. Several issues are closed by maintainers as NOT_PLANNED or "expected behavior"—making runtime detection particularly valuable, since framework-level fixes are not forthcoming.

Detector applicability and firings. Of the 33 mined issues, 17 contain user-posted log excerpts or sufficient narrative detail to assess detector applicability; a released reconstruction script (`experiments/detector_applicability.py`) classified 11/17 (65%) as having signal sufficient to drive a synthetic span sequence. Reconstruction is fault-type dependent: 100% on context-overflow, circular-delegation, and the LangChain guardrail-bypass CVEs (users post token counts, traces, or payloads), but 33%/50% on stale-retrieval/memory-corruption (structural failure modes described behaviorally). Running detectors against the 11 reconstructed traces (Table 10 "Det." column) yields 11/11 detector firings with zero cross-detector false positives, converting prior "feasibility" into a positive end-to-end demonstration on real-world-derived inputs. Per-issue precision/recall on a fully labeled corpus is queued for the journal extension.

5 Related Work

Agent failure taxonomies. MAST [4] identifies 14 failure modes from 1,600+ multi-agent traces with strong inter-annotator agreement ($\kappa = 0.88$); 12 of 14 (85.7%) are detectable via our span

Table 9: Threshold sensitivity sweep on the 112-instance SWE-bench corpus (3,060 spans). Each detector threshold is varied ±50% around its paper default (marked ⋆); other thresholds held fixed. `cost_threshold` is fully insensitive in the studied range (no trace cumulative cost falls between $0.05 and $0.15); `max_retries` and `token_growth_factor` are moderately sensitive and should be calibrated to the workload's expected noise floor.

Threshold	Value	InfRetry	Cost	CtxOver	CircDel	Total
`max_retries`	2	12	0	3	0	15
	3⋆	4	0	3	0	7
	4	2	0	3	0	5
	5	0	0	3	0	3
	6	0	0	3	0	3
`cost_threshold` ($)	0.05	4	0	3	0	7
	0.075	4	0	3	0	7
	0.10⋆	4	0	3	0	7
	0.125	4	0	3	0	7
	0.15	4	0	3	0	7
`token_growth_factor`	1.15	4	0	21	0	25
	1.30⋆	4	0	3	0	7
	1.45	4	0	0	0	4
	1.60	4	0	0	0	4
	2.00	4	0	0	0	4

Table 10: GitHub issue mining: real-world evidence across 6 agent frameworks. All 14 fault types have direct user reports. "Det." column shows detector firings on synthetic spans reconstructed from the 11 reconstructible issues (✓=fires; —=narrative without reconstructible signal); 11/11 detectors fired with zero cross-detector false positives.

Fault Type	Example Issue	Frameworks	Prev.	Det.
infinite_loop	LG #6731, LC #26019	LC, LG, Cr	High	✓
context_overflow	LC #12264, LC #11405	LC, AG	High	✓
circ. delegation	Cr #330	Cr	High	✓
hallucination	OAI #609610	OAI	High	—
guardrail_bypass	LC #21592, NeMo #1485	LC, NeMo	Med	✓
agent_misroute	Cr Forum #3179	Cr	Med	—
tool_failure	LC-AWS #277	LC	Med	✓
timeout	OAI #452505	AG, Cr	Med	—
planning_failure	LG #6731	LG, Cr	Med	—
cost_explosion	(via overflow issues)	All	Med	—
reasoning_loop	(co-occurs w/ inf. loop)	Cr	Med	—
wrong_tool	OAI #609610	OAI	Med	—
stale_retrieval	Cr #3354 (LC)	Cr, LC	Med	✓
memory_corruption	Cr #827, Cr #2753	Cr	Med	✓

LC=LangChain, LG=LangGraph, Cr=CrewAI, AG=AutoGen, OAI=OpenAI, NeMo=NeMo Guardrails. CVEs included for `guardrail_bypass`: CVE-2024-8309 (CVSS 9.8 Critical).

kinds, and the two gaps—failure to ask for clarification and missing verification—are omission failures requiring expected-span specifications. AgentDebug [16] categorizes failures across five categories (memory, reflection, planning, action, system-level), all of which map to our span kinds. Aegis [12] classifies six agent-environment failure modes. These taxonomies characterize *what* fails but provide no runtime detection infrastructure; our benchmark provides the evaluation harness to measure whether a given observability system can *detect* these failures. Post-hoc debugging tools (AgentRx [3], AGDebugger [6]) operate on logs or proprietary formats; AgentTelemetry provides the standardized OTel-native layer they could consume.

LLM observability platforms. LangSmith [8] is LangChain-specific. Langfuse [9] and Datadog LLM capture traces but define no reusable span taxonomy. OpenLLMetry [14] intercepts API calls but cannot distinguish agent-level semantics—a planning step and a summarization call are both `POST /v1/chat/completions`. None define agent-specific span kinds that compose with OTel (Table 8 quantifies the gap on a shared corpus). Enforcement mechanisms such as GuardAgent [15] and NeMo Guardrails [11] are complementary to our `GUARD_RAIL` observability span—enforcement without monitoring leaves a blind spot.

OTel standards. The OTel GenAI conventions [10] define 8 operation types; our taxonomy is a strict superset that adds 5 novel span kinds for agent orchestration phases absent from GenAI. Spans export via OTLP to any GenAI-aware backend, and GenAI attributes coexist with `agenttelemetry.*` attributes. The proposal is submitted to the OpenTelemetry Semantic Conventions repository as PR #3594 [2] for community review.

6 Dataset and Reproducibility

The benchmark is fully reproducible without API keys. Released artifacts include deterministic mock LLM clients (hash-based responses), a fault injector with ground-truth labels for all 14 fault types, 2,940 configuration files, 3,060 SWE-bench spans from 112 instances with span-kind/fault-type labels, the 9×14 ablation script, and the 3,700+ LOC pip-installable Python library (78 tests, all seven adapters). Mock clients ensure bit-exact platform reproducibility.

6.1 Threats to Validity

Internal. Mock-client circularity is mitigated by (1) 85.7% MAST [4] coverage; (2) GitHub-mining confirmation of all 14 fault types and 11/11 detector firings on reconstructed traces (Table 10); (3) the structural ablation argument; and (4) the SWE-bench plus 159-run real-LLM complement. **External / Construct.** Six of seven adapters are manually-instrumented; LangChain's `AgentExecutor` runs end-to-end against real OpenAI APIs (136 spans, 5 kinds, correct cost tracking). The benchmark uses one research-assistant architecture; tree-of-thought, debate, and hierarchical multi-agent generalization remains future work. FDR is binary; the diagnostic-quality reductions (66.8%/68.4%) provide complementary evidence.

7 Conclusion

We presented AgentTelemetry: 14 fault types mapped to 9 span kinds (5 novel agent-orchestration primitives), 490 fault-detection cells, an ablation proving each kind necessary, and a SWE-bench

case study where 84/112 (75%) iteration-exhausted runs partition into reasoning loops invisible to vanilla OTel. A telemetry-guided intervention raises the recovery rate by +12.5 pp (not significant at n=24). Future work: OTel Semantic Conventions integration (PR #3594 [2]), official SWE-bench harness verification, power-justified intervention scaling, and a controlled human study. Toolkit, data, and configurations are open-source.

Data-Availability Statement

The complete AgentTelemetry reproduction package—toolkit source (3,700+ LOC, 78 tests, 7 framework adapters), benchmark harness (2,940 reproducible configurations across 14 fault types × 5 observability conditions × 7 frameworks × 6 mock-LLM seeds), the 9×14 ablation matrix script, the 112-instance SWE-bench Lite case study (3,060 spans with ground-truth labels), the threshold-sensitivity sweep, the 159-run real-LLM validation harness, and the 33-issue GitHub mining corpus—is archived on Zenodo at https://doi.org/10.5281/zenodo.20129006 [1] (version `v0.1.0-aiware2026`, Apache-2.0). The latest version is always reachable via the concept DOI https://doi.org/10.5281/zenodo.20129005. Source repository: https://github.com/Krishnachaitanyakc/AgentTelemetry.

References

[1] Krishna Chaitanya Balusu. 2026. *AgentTelemetry: A Fault Detection Benchmark and Toolkit for LLM Agent Observability (AIware 2026 Reproduction Package).* doi:10.5281/zenodo.20129006

[2] Krishna Chaitanya Balusu. 2026. AI Agent Planning Semantic Conventions. OpenTelemetry Semantic Conventions Pull Request #3594. https://github.com/open-telemetry/semantic-conventions/pull/3594

[3] Shraddha Barke, Anirudh Goyal, Aniket Khare, Aman Singh, Suman Nath, and Chetan Bansal. 2026. AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. arXiv:2602.02475 doi:10.48550/arXiv.2602.02475

[4] Mert Cemri, Melissa Z. Pan, Shuyi Yang, et al. 2025. Why Do Multi-Agent LLM Systems Fail?. In *Advances in Neural Information Processing Systems (NeurIPS).* doi:10.48550/arXiv.2503.13657

[5] Liming Dong, Qinghua Lu, and Liming Zhu. 2024. AgentOps: Enabling Observability of LLM Agents. arXiv:2411.05285 doi:10.48550/arXiv.2411.05285

[6] Will Epperson, Gagan Bansal, Victor Dibia, Adam Fourney, Jacob Gerrits, Erkang Zhu, and Saleema Amershi. 2025. Interactive Debugging and Steering of Multi-Agent AI Systems. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI).* doi:10.1145/3706598.3713581

[7] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *Proceedings of the 12th International Conference on Learning Representations (ICLR).* https://openreview.net/forum?id=VTF8yNQM66

[8] LangChain Inc. 2024. LangSmith: Production-Grade LLM Platform. https://smith.langchain.com/

[9] Langfuse GmbH. 2024. Langfuse: Open-Source LLM Observability. https://langfuse.com/

[10] OpenTelemetry Authors. 2024. OpenTelemetry Specification v1.30. https://opentelemetry.io/docs/specs/otel/

[11] Traian Rebedea, Razvan Dinu, Makesh Sreedhar, Christopher Parisien, and Jonathan Cohen. 2023. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP Demo).* doi:10.18653/v1/2023.emnlp-demo.40

[12] Kevin Song, Anand Jayarajan, Yaoyao Ding, et al. 2025. Aegis: Taxonomy and Optimizations for Overcoming Agent-Environment Failures in LLM Agents. arXiv:2508.19504 doi:10.48550/arXiv.2508.19504

[13] Anselm Strauss and Juliet Corbin. 1998. *Basics of Qualitative Research* (2nd ed.). Sage Publications.

[14] Traceloop. 2024. OpenLLMetry: Open-Source LLM Observability. https://github.com/traceloop/openllmetry

[15] Zhen Xiang, Linzhi Zheng, Yanjie Li, et al. 2025. GuardAgent: Safeguard LLM Agents by a Guard Agent via Knowledge-Enabled Reasoning. In *Proceedings of the 42nd International Conference on Machine Learning (ICML).* doi:10.48550/arXiv.2406.09187

[16] Kunlun Zhu, Zheng Liu, Bo Li, et al. 2025. Where LLM Agents Fail and How They Can Learn From Failures. arXiv:2509.25370 doi:10.48550/arXiv.2509.25370

Received 2026-02-15; accepted 2026-03-28